

A Lean 4 Certificate of Scott's Data Types as Lattices (1976) generated by LLM and registered by Palomar

Lars Warren Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 12, 2026

Github: <https://github.com/catskillsresearch/scott1976>

Palomar Registration:

<https://palomar-registry.org/entry?id=PALOMAR-2026-08-25-000004&version=1>

Primary Category: cs.LO (Logic in Computer Science)

Secondary Category: math.LO (Logic)

Abstract

This note records a Lean 4 / mathlib formalization of Dana Scott's 1976 paper *Data Types as Lattices*. Scott develops the graph model of the λ -calculus inside the complete lattice $\mathbf{P}\omega$ of subsets of the natural numbers, then uses retracts of that lattice as data types. The Lean development is a 1:1 onto translation of every numbered definition, theorem, displayed equation used as a definition or example, and Tables 1–3.

The sorry-free library consists of approximately 17,000 lines in 15 files under `Scott1976/` (the root import plus fourteen modules). Palomar Challenge / Solution files select the 63 theorem declarations and 70 locked definitions named in `comparator.json`. A Comparator match establishes those selected statements, not every informal clause of every numbered theorem; further companions remain in the library. The locked coproduct `plusR` uses the doubly strict conditional of (4.4) rather than the ordinary conditional written in displayed (4.10).

The repository has no project-defined axioms and no Lake dependency beyond mathlib. Completed proofs use the standard classical mathlib footprint `[propext, Quot.sound, Classical.choice]`. Deliberate proof holes occur only in the Mathlib-only `Challenge.lean`; `Solution.lean` re-exports the kernel-checked development.

Dana Scott did not review or endorse this formalization. Lean code was written by AI agents under the direction and review of the author, who takes sole responsibility for the mathematical content.

Proofs in this note are summarized at the level of their main constructions. Short representative Lean fragments are copied from the library. The Appendix gives a short description of each library file and a link to that file in the Palomar-preserved copy of this repository.

1 Introduction

Scott's monograph answers a single question in several registers: what is a data type, if types are to be lattices and programs are to be continuous maps? The carrier is $\mathbf{P}\omega$, the power set of $\mathbb{N}$, ordered by inclusion. Finite subsets e_n are a countable basis. A map $f : \mathbf{P}\omega \rightarrow \mathbf{P}\omega$ is continuous when it preserves directed unions, equivalently when $f(x) = \bigcup \{f(e_n) \mid e_n \subseteq x\}$. Continuous

maps are encoded as graphs and recovered by application, so the function space lives inside $\mathbf{P}\omega$ itself.

LAMBDA is the combinatory language of that encoding. Its interpretation is continuous in every free variable (Theorem 2.3), the compared conversion theorem is a conditional semantic congruence (Theorem 2.2), and a continuous map is computable exactly when its graph is recursively enumerable, equivalently combinatory, equivalently LAMBDA-definable (Theorem 2.6). Enumeration then internalizes Gödel numbering: `val` enumerates the combinatory subalgebra $\mathbf{RE}$, the second recursion theorem and Myhill–Shepherdson appear as continuity facts, and degrees of unsolvability are finitely generated combinatory subalgebras.

Retracts of $\mathbf{P}\omega$ are the data types. The constructions $a \circ \rightarrow b$, $a \otimes b$, and $a \oplus b$ are themselves retracts when a and b are, inverse limits exist by the fixed-point theorem, and recursive domain equations such as

$$\mathbf{lamb} = \mathbf{int} \oplus (\mathbf{lamb} \circ \rightarrow \mathbf{lamb}), \quad \mathbf{exp} = \mathbf{int} \oplus \mathbf{nil} \oplus \mathbf{exp} \oplus \mathbf{exp} \oplus (\mathbf{exp})^4 \oplus (\mathbf{exp} \otimes \mathbf{exp}) \oplus (\mathbf{int} \otimes \mathbf{exp})$$

are solved in place. Closures classify algebraic lattices; V is a universe of closures. The Borel/analytic hierarchy of §§6–7 is expressed with continuous preimages and equalizers, and restricted equivalences make the combinators I, K, S and the iterators functional.

The formalization follows this mathematical decomposition rather than treating the forty-odd numbered theorems as unrelated endpoints.

2 Scope and layout

The library has 15 Lean files and approximately 17,000 lines. The root module `Scott1976.lean` imports the complete development; `Basic.lean` is a thin facade. `Solution.lean` imports `Scott1976/*`.

Module	Role
<code>Pomega</code>	$\mathbf{P}\omega$, e_n , $\perp$, $\top$, pairing
<code>Continuous</code>	Scott-open sets, continuity, Theorem 1.1
<code>Graph</code>	<code>graph</code> / <code>fun</code> , Theorem 1.2
<code>FixedPoint</code>	Substitution, least fixed points, Theorems 1.3–1.4
<code>Embedding</code>	Extension and embedding, Theorems 1.5–1.6
<code>Lambda</code>	Table 1–2, combinators, Theorems 2.1–2.5
<code>Computability</code>	r.e. graphs, Theorem 2.6 (unary and k -ary)
<code>Enumeration</code>	$\mathbf{G}$, <code>val</code> , Theorems 3.1–3.7, <code>vaal</code>
<code>FixedLattices</code>	Algebraic and continuous lattices (support)
<code>Retracts</code>	Retracts, $\circ \rightarrow$, $\otimes$, $\oplus$, Theorems 4.1–4.6, <code>exp</code>
<code>Functionality</code>	(4.38)–(4.45) <code>tree/lamb/H</code> , Theorems 7.1–7.4
<code>Closures</code>	Closures, V , Theorems 5.1–5.6
<code>Classification</code>	$\mathfrak{G}, \mathfrak{F}, \mathfrak{B}$, Theorems 6.1–6.7, Table 3
<code>Challenge / Solution</code>	Palomar statement of record and sorry-free realization

The published scope is the numbered items of §§1–7 together with Tables 1–3. One source-facing note remains visible in the inventory and is not a missing proof: Theorem 4.5’s informal coproduct non-uniqueness remark is omitted.

The project is standalone. It imports none of the sibling formalizations of Scott’s later domain-theory papers. Cross-presentation equivalences live in the separate `scott_models` repository.

3 How the proofs use mathlib

Mathlib supplies the set-theoretic and classical substrate. The predicates and reductions specific to Scott’s paper are defined in this repository.

Power sets and directed unions. $\mathbf{P}\omega$ is $\mathbf{Set} \ \mathbb{N}$. Directed suprema are ordinary unions of directed families; the finite basis e_n is an explicit pairing enumeration. Continuity is the equality $f(x) = \bigcup_n f(e_n)$ on $e_n \subseteq x$, not a topology imported from mathlib’s Scott topology.

Classical graphs. Application `funOf u x` decodes pairs from the graph of u . Least fixed points are $\bigcup_n f^n(\emptyset)$. The only axioms that appear are mathlib’s standard classical triple.

Recursion and combinators. Syntax of LAMBDA and of the modified language `LambTerm` is an inductive type. Interpretation lemmas are ordinary structural induction. Primitive-recursive arithmetic on codes uses mathlib $\mathbb{N}$ lemmas; r.e. sets are ranges of combinatory enumerations, not an imported computability library.

Retracts as idempotents. A retract is a graph a with $a = a \circ a$. Typedness $x : a$ is the equation $x = a(x)$. Inverse limits of retracts are constructed from the iterates of a continuous functor by Theorem 1.4 / Theorem 4.6, not from a categorical limit API.

What mathlib does not supply is Scott’s interface: graphs of continuous maps on $\mathbf{P}\omega$, LAMBDA conversion, the combinatory algebra **RE**, retract constructors $\circ \rightarrow / \otimes / \oplus$, closures and V , or the $\mathfrak{G} / \mathfrak{F} / \mathfrak{B}$ classification. Those constructions are developed here.

4 Proof dependency structure

The published development is one lattice-theoretic core and six applications (Figure 1).

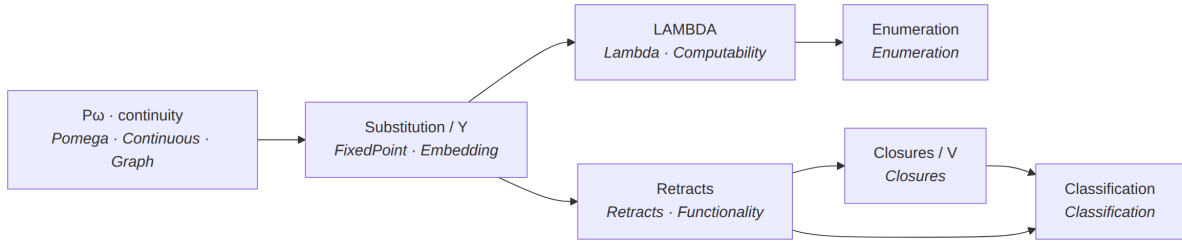


Figure 1: Dependency of the lattice-theoretic core and its six applications.

Section 4’s domain examples depend on the retract constructors and on the fixed-point theorem. The continuous decoder $\mathcal{H}$ of (4.45) is the least fixed point of a syntax-directed operator, then retracted onto $\mathbf{exp} \circ \rightarrow (\mathbf{env} \circ \rightarrow \mathbf{lamb})$ (Figure 2):

Enumeration and the tree evaluator (4.46) are a parallel use of the same fixed-point pattern (Figure 3):

Closures reuse retracts but add $I \subseteq a$ (Figure 4):

5 Living inventory

Status words:

- **faithful** — Lean statement matches Scott; library proof is sorry-free.
- **partial** — a named Lean declaration exists but is weaker than a tempting modern strengthening; the Lean statement is paper-accurate.
- **missing** — no faithful Lean declaration yet.

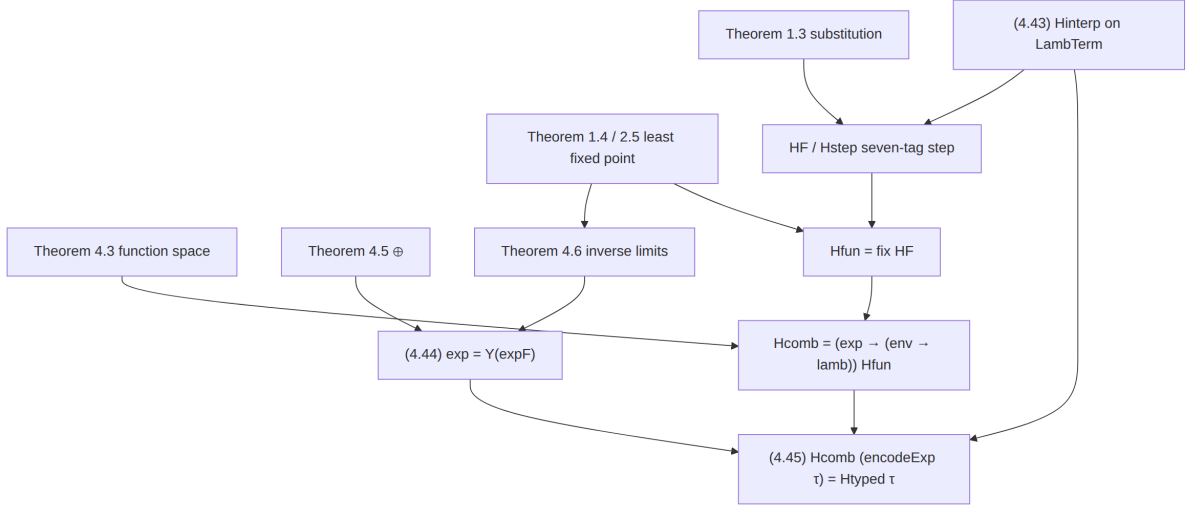


Figure 2: Proof dependencies for the continuous decoder $\mathcal{H}$ of (4.45).

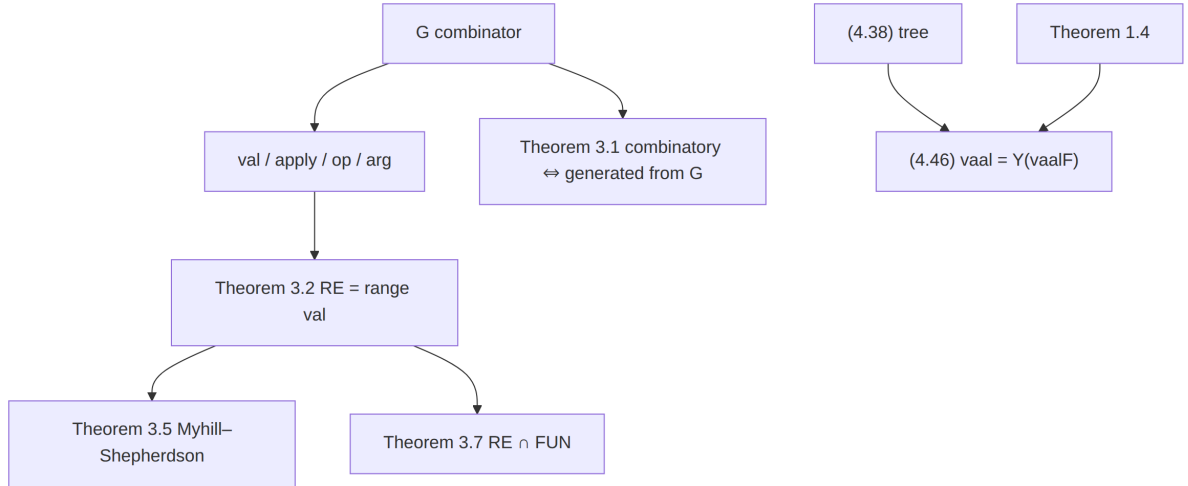


Figure 3: Enumeration and the tree evaluator (4.46).

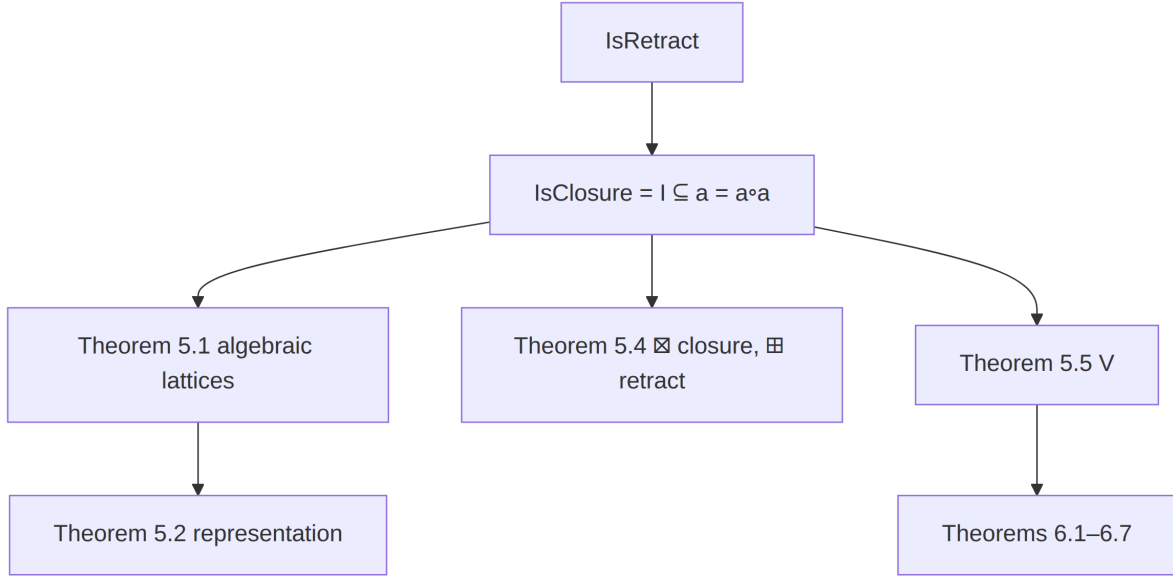


Figure 4: Closures, V , and the classification theorems.

The library inventory below is **faithful** except for the Palomar-selected specializations and strengthenings recorded in **formalization.yaml** limitations. Comparator selects 63 theorems and 70 definitions from **comparator.json**; it does not claim that those names exhaust every informal clause of every numbered theorem.

Compared numbered theorems and selected companions:

- §1: theorem_1_1–theorem_1_6 (with theorem_1_3_tuple, theorem_1_3_nary, theorem_1_5_extends)
- §2: theorem_2_1–theorem_2_6 (with theorem_2_4_complete, theorem_2_6_fin)
- §3: theorem_3_1–theorem_3_7 (with theorem_3_2_range)
- §4: theorem_4_1–theorem_4_6 (with theorem_4_1_continuous, theorem_4_4_typed, tensorR_isRetract, tensorR_isStrict, tensorR_retractLe, tensorR_functor, theorem_4_5_sum, plusR_retractLe, plusR_functor, theorem_4_6_limit)
- §5: theorem_5_1–theorem_5_6 (with theorem_5_3_closure, typed_squarePair_boxTensor, theorem_5_4_plus, boxPlus_typed_iff, theorem_5_5_universe, theorem_5_6_combinator)
- §6: theorem_6_1–theorem_6_7
- §7: theorem_7_1–theorem_7_4

5.1 §1 Continuous functions

Item	Lean	Status
Def. $P\omega$, e_n , $\perp$, $\top$, pair	$P\omega$, e , botElem , topElem , pair	faithful
Def. basic neighbourhood / Scott-open	basicNhhd , IsScottOpen	faithful

Item	Lean	Status
Def. continuity $f(x) = \bigcup f(e_n)$	IsScottContinuous	faithful
Def. graph, fun	graph, funOf	faithful
Theorem 1.1	theorem_1_1	faithful
Theorem 1.2	theorem_1_2	faithful
Theorem 1.3	theorem_1_3, theorem_1_3_tuple, theorem_1_3_nary	faithful — unary/binary/ternary substitution, not arbitrary finite arity
Theorem 1.4	theorem_1_4	faithful
Theorem 1.5	theorem_1_5, theorem_1_5_extends	theorem_1_5 is a strengthening (continuity of <code>extend f</code> for every <code>f</code>); <code>theorem_1_5_extends</code> restores Scott's continuity hypothesis
Theorem 1.6	theorem_1_6	faithful — countably based T_0 spaces

5.2 §2 LAMBDA

Item	Lean	Status
Table 1 $(\alpha)(\beta)(\xi)$	theorem_2_2_alpha, theorem_2_2_beta, theorem_2_2_xi	faithful
Table 1 (η) fails	eta_fails	faithful
Table 1 $(\xi^*)(\mu)$	xi_star, table1_mu / mu_law	faithful
Table 2 syntax/semantics	Term, interp	faithful
(2.1)(2.2) max/min extension	maxExtend, minExtend, eq_2_1_empty/_singleton/_else	faithful
(2.3)(2.4) relation/sum	relComp, setSum	faithful
(2.5)(2.6)(2.7) examples	eq_2_5, eq_2_6, eq_2_7_bot/_zero/_succ/_pos/_mix	faithful
(2.8) Y	Ycomb, Ycomb_unfold	faithful
(2.9)–(2.13) distribution	eq_2_9 ... eq_2_13, eq_2_11_graphs	faithful — (2.11) equality when both arguments are graphs
(2.14)–(2.16)	eq_2_14, eq_2_15, union_via_cond, eq_2_16	faithful
FUN	FUN	faithful
(2.17) intersection encoding	eq_2_17, eq_2_17_step, interC	faithful
(2.18) K-fixed points	eq_2_18	faithful
(2.19)–(2.23) sequences	seq0, seq1, seq2, seqCons, eq_2_22, eq_2_23_*	faithful

Item	Lean	Status
(2.24)–(2.27) $\$, \text{primrec}$	<code>eq_2_24, eq_2_24_Y, eq_2_24_Ycomb, dollarC, eq_2_25, lamOmega, eq_2_27, primRecVal_fix</code>	faithful — $\$ = Y(\text{step})$; $\hat{p} = \$(Y(\text{primRecStep}))$
(2.28) Y commuting	<code>eq_2_28, eq_2_28_commute</code>	faithful
Theorem 2.1	<code>theorem_2_1</code>	faithful
Theorem 2.2	<code>theorem_2_2</code>	partial — compared form is conditional semantic congruence (renamed bodies already agree), not source-level alpha from freshness
Theorem 2.3	<code>theorem_2_3, _unary, _binary, _ternary</code>	faithful — any finite arity via nested graphs
Theorem 2.4	<code>theorem_2_4, theorem_2_4_complete, erase</code>	faithful
Theorem 2.5	<code>theorem_2_5</code>	faithful
Def. computable	<code>IsComputable</code>	faithful
Theorem 2.6	<code>theorem_2_6, theorem_2_6_fin, IsComputableFin</code>	faithful — unary and k-ary; the nested graph is the pairing encoding of $m \in f(e_{n_0}) \cdots (e_{n_{k-1}})$

5.3 §3 Enumeration

Item	Lean	Status
G	<code>Gcomb, Gcomb_app_self</code>	faithful — $G(G)=0$; $G(\perp)=\perp$
(3.1)(3.2)	<code>eq_3_1, eq_3_2</code>	faithful
Theorem 3.1	<code>theorem_3_1, GeneratedFromG</code>	faithful — combinatory iff generated from G alone
(3.4)–(3.7) $\text{apply}, \text{op}, \text{arg}, \text{val}$	<code>applyNat, opNat, argNat, valNat</code>	faithful — Scott's $\text{val}(0)=G$, $\text{apply}=(n,m)+1$
Theorem 3.2	<code>theorem_3_2, theorem_3_2_range</code>	faithful — $\text{RE} = \text{range val}$ with (i)(ii)
(3.8)–(3.12)	<code>eq_3_8_* \dots eq_3_12</code>	faithful
fin	<code>fin, valNat_fin</code>	faithful — $\text{val}(\text{fin } j)=e_j$
Theorem 3.3	<code>theorem_3_3, theorem_3_3_val</code>	faithful — $\text{primrec } v/\text{rec}$ with (i)–(iii)
Theorem 3.4	<code>theorem_3_4</code>	faithful — $\neg \text{IsRE } \{n \mid \text{val } n = \perp\}$
Theorem 3.5	<code>theorem_3_5, myhillQ</code>	faithful — LAMBDA-definable extensional p yields LAMBDA-definable q with $\text{val}(p \ n) = q(\text{val } n)$

Item	Lean	Status
Theorem 3.6	<code>theorem_3_6</code> , <code>theorem_3_6_generated</code>	faithful — enumeration degrees are exactly the finitely generated combinatory subalgebras
R,L	<code>Rcomb</code> , <code>Lcomb</code> , <code>Rcomb_app</code> , <code>Lcomb_app</code>	faithful
(3.15)–(3.17)	<code>barComb</code> , <code>eq_3_15</code> , <code>eq_3_16</code> , <code>eq_3_17</code>	faithful — $\bar{u} = Y(\text{barStep})$; operator identities (3.16) and (3.17)
Theorem 3.7	<code>theorem_3_7</code>	faithful — $\text{RE} \cap \text{FUN}$ is the semigroup generated by R, L, and $\tilde{G}$

5.4 §4 Retracts

Item	Lean	Status
Def. <code>retract</code> , $u:a, \sqsubseteq$ (4.1)–(4.10) combinators	<code>IsRetract</code> , <code>typed</code> , <code>retractLe</code> , <code>funRetract</code> , <code>arrowR</code> , <code>tensorR</code> , <code>plusR</code> , <code>dcondSet</code> , <code>boolR</code> , <code>intR</code> , ...	faithful <code>plusR</code> is a justified modification of displayed (4.10): Scott writes ordinary $\supset$, Lean uses the doubly strict $\sqsupset$ of (4.4), as in <code>bool</code> (4.3), so mixed tags map to $\top$ and Theorem 4.5 ($\oplus$ is a retract) holds; not a symbol-for-symbol 1:1 of the displayed equation. (4.7) <code>int</code> is $Y(\text{intF})$ with Scott's three-case table
Theorem 4.1	<code>theorem_4_1</code> , <code>theorem_4_1_complete</code> , <code>theorem_4_1_continuous</code>	faithful — fixed points of a continuous map form a complete lattice; retract ranges are continuous lattices
Theorem 4.2	<code>theorem_4_2</code>	faithful
Theorem 4.3	<code>theorem_4_3</code> , <code>arrowR_isRetract</code>	faithful — (i)–(v) packaged
Theorem 4.4	<code>theorem_4_4</code> , <code>theorem_4_4_typed</code> , <code>theorem_4_4_med</code> , <code>theorem_4_4_med_unique</code> , <code>tensorR_isRetract</code> , <code>tensorR_isStrict</code> , <code>tensorR_retractLe</code> , <code>tensorR_functor</code> , <code>eq_4_12</code> , <code>eq_4_17</code> – <code>eq_4_21</code> , <code>eq_4_24</code> , <code>eq_4_25</code> , <code>eval_curry</code>	faithful — product, mediator, CCC <code>eval/curry</code>

Item	Lean	Status
Theorem 4.5	<code>theorem_4_5_sum</code> , <code>plusR_isRetract</code> , <code>plusR_typed_iff</code> , <code>plusR_retractLe</code> , <code>plusR_functor</code> , <code>eq_4_13</code> , <code>eq_4_32</code> – <code>eq_4_37</code> , <code>inleftC</code> , <code>inrightC</code> , <code>outleftC</code> , <code>outrightC</code> , <code>whichC</code> , <code>outC</code> , <code>boolR_isRetract</code>	faithful — (i)–(iv) packaged; coproduct non-uniqueness remark omitted
Theorem 4.6	<code>theorem_4_6</code> , <code>theorem_4_6_limit</code>	<code>theorem_4_6_limit</code> is the order-isomorphism (inclusion-preserving bijection) with the inverse limit, not a homeomorphism
(4.38) <code>tree</code>	<code>treeR</code> , <code>eq_4_38</code> , <code>tree_atom</code> , <code>tree_node</code>	faithful — <code>tree</code> = <code>nil</code> $\oplus$ (<code>tree</code> $\otimes$ <code>tree</code>); <code>atom</code> and binary nodes
(4.39)–(4.43) <code>lamb</code>	<code>lambR</code> , <code>envR</code> , <code>updateEnv</code> , <code>LambTerm</code> , <code>Hinterp</code>	faithful — data types and $\mathcal{H}$ by recursion on finite terms
(4.44) <code>exp</code>	<code>expR</code> , <code>eq_4_44</code> , <code>expR_isRetract</code>	faithful — seven-tag syntax functor; <code>exp</code> = $Y(\text{expF})$ is a retract
(4.45) $\mathcal{H}$	<code>eq_4_45</code> , <code>HF</code> , <code>Hfun</code> , <code>Hcomb</code> , <code>Htyped</code> , <code>Hcomb_decode</code>	faithful — $\mathcal{H}$ is the retract of the least fixed point of a continuous seven-tag operator; <code>Hcomb</code> (<code>encodeExp</code> τ) = <code>Htyped</code> τ
(4.46) <code>vaal</code>	<code>vaalC</code> , <code>vaalC_unfold</code> , <code>vaalC_combinatorial</code> , <code>vaalC_typed</code>	faithful — <code>vaal</code> : <code>tree</code> $\rightarrow$ <code>id</code> as the least fixed point of (4.46), <code>combinatorial</code> and <code>typed</code>

5.5 §5 Closures

Item	Lean	Status
Def. closure $I \subseteq \mathbf{a} = \mathbf{a} \circ \mathbf{a}$	<code>IsClosure</code>	faithful
(5.1)–(5.13) <code>pairing/box</code>	<code>eq_5_1</code> , <code>eq_5_2</code> , <code>squarePair</code> , <code>boxTensor</code> , <code>boxPlus</code> , <code>funRetract_isClosure</code> , <code>bool</code>	faithful
Theorem 5.1	<code>theorem_5_1</code> , <code>theorem_5_1_isolated</code>	faithful — closure fixed points form an algebraic lattice; compacts are the $\mathbf{a}(\mathbf{e} \ \mathbf{n})$
Theorem 5.2	<code>theorem_5_2</code>	faithful — every countable algebraic lattice is order-isomorphic to the range of a closure

Item	Lean	Status
Theorem 5.3	<code>theorem_5_3</code> , <code>theorem_5_3_closure</code> , <code>Icomb_subset_arrowR</code>	faithful
Theorem 5.4	<code>theorem_5_4</code> , <code>typed_squarePair_boxTensor</code> , <code>theorem_5_4_plus</code> , <code>boxPlus_typed_iff</code> , <code>eq_5_12</code> , <code>eq_5_22</code> , <code>eq_5_23</code>	faithful — $\boxtimes$ and $\boxplus$ of closures are closures; compared typing analogues of Theorems 4.4–4.5 are <code>typed_squarePair_boxTensor</code> and <code>boxPlus_typed_iff</code>
Theorem 5.5	<code>theorem_5_5</code> , <code>theorem_5_5_iff</code> , <code>theorem_5_5_universe</code> , <code>Vcomb</code> , <code>eq_5_14</code> , <code>eq_5_19</code> , <code>eq_5_21</code>	faithful — V is a closure; $V(a)=a$ iff a is a closure; (5.14)=(5.15)
Theorem 5.6	<code>theorem_5_6</code> , <code>theorem_5_6_iterates</code> , <code>eq_5_24</code>	faithful — $Y(f):V$ whenever $f:V \circ \rightarrow V$; $\circ \rightarrow$ typed on V
(5.25) $d = I \cup (d \circ \rightarrow d)$	<code>dEq</code> , <code>eq_5_25</code>	faithful — $Y(\lambda a. I \cup (a \circ \rightarrow a))$; $d = d \circ \rightarrow d$ if $I \subseteq d \circ \rightarrow d$

5.6 §6 Classification

Item	Lean	Status
Def. $\mathfrak{G}, \mathfrak{F}, \mathfrak{B}$	<code>ScottG</code> , <code>ScottF</code> , <code>ScottB</code>	faithful
Theorem 6.1	<code>theorem_6_1</code>	faithful
Theorem 6.2	<code>theorem_6_2</code>	faithful
Theorem 6.3	<code>theorem_6_3</code>	faithful
Theorem 6.4	<code>theorem_6_4</code>	faithful
Theorem 6.5	<code>theorem_6_5</code>	partial — specialization to the representative target <code>succSet topElem</code>
Theorem 6.6	<code>theorem_6_6</code>	faithful
Theorem 6.7	<code>theorem_6_7</code>	faithful — B_δ sets are exactly equalizers of continuous maps
Table 3 typical sets	<code>typicalG ... typicalPi11</code> , <code>typical*_universal</code>	faithful — each class is the continuous preimages of the typical set

5.7 §7 Functionality

Item	Lean	Status
Def. restricted equivalence	<code>RestrictedEquiv</code>	faithful
(7.4)–(7.8) $E_a, \rightarrow, \times, +$	<code>Ea</code> , <code>arrowE</code> , <code>prodE</code> , <code>sumE</code>	faithful
Theorem 7.1	<code>theorem_7_1</code>	faithful

Item	Lean	Status
Theorem 7.2	<code>theorem_7_2</code>	faithful — E_a a is the identity on the range of a and is isomorphic to the kernel $\ker E_a$; $E_{\{a \circ \rightarrow b\}} \cong E_a \rightarrow E_b$, $E_{\{a \otimes b\}} = E_a \times E_b$, and the $\oplus$ identity with $\{\perp, \top\}$
Theorem 7.3	<code>theorem_7_3</code>	faithful
Theorem 7.4	<code>theorem_7_4</code> , <code>theorem_7_4_</code> <code>unique</code>	faithful — Z_n : $(A \rightarrow A) \rightarrow (A \rightarrow A)$ for all A , and this with $z(f) = z(\lambda x. f(x))$ characterizes the iterators
(7.15)–(7.19) iterators	<code>Z</code> , <code>Zcomb</code> , <code>sigmaJ</code> , <code>eq_7_18_*</code>	faithful

6 Proof notes

6.1 Theorems 1.1–1.6 — continuity, graphs, and fixed points

Theorem 1.1 characterizes continuity by preservation of the finite basis. Theorem 1.2 says that u is a graph if and only if $u = \text{graph}(\text{fun } u)$. Substitution (Theorem 1.3) is used throughout as composition and as binary and ternary substitution on the diagonal (`theorem_1_3_tuple`, `theorem_1_3_nary`); the compared forms are not arbitrary finite-arity substitution:

Lean 4 source

```
theorem theorem_1_3 {f g : Pomega -> Pomega}
  (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
  IsScottContinuous (fun x => f (g x))
```

The least-fixed-point theorem is the engine of every recursive domain equation and of (4.45):

Lean 4 source

```
theorem theorem_1_4 {f : Pomega -> Pomega} (hf : IsScottContinuous f) :
  f (fix f) = fix f /\ forall x, f x = x -> fix f subseq x
```

Theorem 1.5 is compared as a strengthening: `extend f` is continuous for every f , not only continuous f ; `theorem_1_5_extends` restores Scott’s continuity hypothesis for agreement on the subspace. Theorem 1.6 embeds a countably based T_0 space into $\mathbf{P}\omega$ along a countable basis.

6.2 Theorems 2.1–2.6 — LAMBDA and computability

Table 1 is proved as equational laws of `interp`, including the failure of (η) . The compared `theorem_2_2` is a conditional semantic congruence: it assumes the renamed bodies already have equal interpretations and lifts that equality through `lambda`; it is not source-level α -conversion from a freshness hypothesis. Theorem 2.3 is the finite-arity substitution theorem: a continuous map of n arguments is represented by a nested graph.

Lean 4 source

```
theorem theorem_2_3 {n : Nat} {f : (Fin n -> Pomega) -> Pomega}
  (hf : IsScottContinuousFin f) :
  exists u, forall xs, nestApplyFin u xs = f xs
```

Theorem 2.4 closes the combinatory algebra generated by the six constants 0, succ, pred, cond, K , and S . Theorem 2.5 identifies Y with the least-fixed-point combinator. Theorem 2.6 is the computability trinity. The Palomar-locked unary form is

Lean 4 source

```
theorem theorem_2_6 {f : Pomega -> Pomega} (hf : IsScottContinuous f) :
  (IsComputable f <-> IsRE (graph f)) /\
  (IsRE (graph f) <-> IsCombinatory (graph f)) /\
  (IsLambdaDefinable (graph f) <-> IsCombinatory (graph f))
```

The k -ary companion `theorem_2_6_fin` uses the nested-graph encoding of $m \in f(e_{n_0}) \cdots (e_{n_{k-1}})$.

6.3 Theorems 3.1–3.7 — enumeration

$\mathbf{G}$ is a combinator with $G(G) = 0$ and $G(\perp) = \perp$. Theorem 3.1 says the combinatory elements are exactly those generated from G alone. The enumeration `val` is Scott's: `val(0)=G` and `apply(n,m)=(n,m)+1`. Theorem 3.2 identifies $\mathbf{RE}$ with the range of `val`. The second recursion theorem, incompleteness of $\{n \mid \text{val } n = \perp\}$, Myhill–Shepherdson, enumeration degrees, and the semigroup $\mathbf{RE} \cap \mathbf{FUN}$ are Theorems 3.3–3.7.

The tree evaluator (4.46) is the same pattern on the retract `tree` rather than on Gödel numbers: `vaalC = fix vaalF` for a continuous step, hence combinatory and typed `tree` $\rightarrow$ `id`.

6.4 Theorems 4.1–4.6 — retracts

A retract is an idempotent graph. Theorem 4.1: the fixed points of a continuous map form a complete lattice (`theorem_4_1`); the range of a retract has a directed way-below basis of finite images (`theorem_4_1_continuous`), which is the compared continuous-lattice half. Theorems 4.3–4.5 package the function-space, product, and sum constructors, including retractness, strictness, preservation of $\sqsubseteq$, the CCC structure `eval/curry`, and the four sum identities. Theorem 4.6 is the inverse-limit theorem used for `int`, `tree`, `lamb`, and `exp`. The compared `theorem_4_6_limit` is an inclusion-preserving bijection with the inverse limit (an order isomorphism), not a homeomorphism:

Lean 4 source

```
theorem theorem_4_6 {F : Pomega -> Pomega}
  (hf : IsScottContinuous F)
  (hret : forall a, IsRetract a -> IsRetract (F a)) :
  (forall n, IsRetract (iterateBot F n)) /\
  IsRetract (funOf Ycomb (graph F))
```

The seven-tag syntax functor of (4.44) is

Lean 4 source

```
def expF (z : Pomega) : Pomega :=
  plus7 intR botElem z z (tensor4 z z z z) (tensorR z z) (tensorR intR z)

def expR : Pomega := funOf Ycomb (graph expF)
```

and `eq_4_44` is the unfolding `expR = expF expR`.

6.5 Equation (4.45) — the continuous decoder $\mathcal{H}$

Scott asks why $\mathcal{H}$ exists, and answers: because of the fixed-point theorem. Rewrite (4.43) by cases on the abstract syntax (4.44). The resulting operator on $\mathbf{P}\omega^{\mathbf{exp}}$ is continuous, so it has a least fixed point, and

$$(4.45) \quad \mathcal{H} : \mathbf{exp} \rightarrow (\mathbf{env} \rightarrow \mathbf{lamb})$$

is computable.

An earlier packaging defined `Hcomb` as the retract of the *union of the graphs of the typed interpreters* `Htyped` τ . That object is typed $\mathbf{exp} \rightarrow (\mathbf{env} \rightarrow \mathbf{lamb})$, but it is not a continuous decoder: `encodeExp` τ is an infinite graph, so `funOf` of a union of finite-support graphs need not recover `Htyped` τ .

The completed construction follows (4.46) and the paper’s Y-argument. One continuous step `Hstep` H e t dispatches on the (4.44) tag of e with the same seven-way `dcondSet` used by `plus7`. Recursive calls are `hApply` H e $t = (H \cdot e) \cdot t$. The abstraction clause cannot test `m = idx` in the index, because equality of codes is not Scott-continuous in `idx`. Instead the environment update is extended by `minExtend`:

Lean 4 source

```
def updateEnvUnion (t x idx : Pomega) : Pomega :=
  minExtend (fun n => updateEnv t x n) idx
```

On a singleton index this is Scott’s $t[x/n]$:

Lean 4 source

```
theorem updateEnvUnion_ofNat (t x : Pomega) (n : Nat) :
  updateEnvUnion t x (ofNat n) = updateEnv t x n
```

The operator and its least fixed point are

Lean 4 source

```
def HF (H : Pomega) : Pomega :=
  graph (fun e => graph (fun t => Hstep H e t))

def Hfun : Pomega := fix HF

def Hcomb : Pomega :=
  funOf (arrowR expR (arrowR envR lambR)) Hfun
```

`HF` is continuous in H because each clause is continuous in the recursive calls (Theorem 1.3). Therefore `Hfun` = `HF Hfun`. Induction on `LambTerm` gives `Hfun` (`encodeExp` τ) t = `Hinterp` τ t , hence `Hfun` (`encodeExp` τ) = `graph` (`Hinterp` τ). Encoded terms are typed on $\mathbf{exp}$, so retracting onto the function space decodes them:

Lean 4 source

```
theorem eq_4_45 (tau : LambTerm) :
  typed (encodeExp tau) expR /\
  typed (Htyped tau) (arrowR envR lambR) /\
  funOf Hcomb (encodeExp tau) = Htyped tau
```

A closed combinatory term for `HF` (in the style of `vaalStepTerm`) is not required for (4.45): the paper’s existence argument is Y of a continuous operator, and `Hcomb` is that least fixed point viewed as a map of retracts. Computability of $\mathcal{H}$ is then Theorem 2.6 plus continuity of `HF`.

6.6 Theorems 5.1–5.6 — closures and V

A closure is a retract containing I . Theorem 5.1: the fixed points form an algebraic lattice whose compact elements are the $a(e_n)$. Theorem 5.2 represents every countable algebraic lattice as the range of a closure. Theorem 5.3 places $\circ \rightarrow$ among closures. Theorem 5.4: the product

$\boxtimes$ and the sum $\boxplus$ of closures are closures, and both are typed on V . The displayed (5.13) is the occupancy-tag formula $([u]_0 \supset 0, 0) \cup ([u]_1 \supset 1, 1) \sqsubset [a'([u]_0), \perp], [\perp, b'([u]_1)]$, not the earlier OCR misread that dispatched on $[u]_0$ alone.

Lean 4 source

```
theorem theorem_5_4_plus {a b : Pomega} (ha : IsClosure a) (hb : IsClosure b) :
  IsClosure (boxPlus a b)
```

Theorem 5.5: V is a closure, and $V(a) = a$ if and only if a is a closure. Theorem 5.6: Y and $\circ \rightarrow$ are typed on $\$V$.

6.7 Theorems 6.1–6.7 — classification

The classes $\mathfrak{G}$, $\mathfrak{F}$, $\mathfrak{B}$ and their δ -refinements are defined by continuous preimages of typical sets (Table 3). Compared `theorem_6_5` is Scott's $\mathfrak{F} \cap \mathfrak{G}_\delta$ characterization specialized to the representative target `succSet topElem`, not the fully parameterized form over every infinite proper subset of $\mathbb{T}$. Theorem 6.7 is the equalizer characterization of $\mathfrak{B}_\delta$:

Lean 4 source

```
theorem theorem_6_7 :
  forall U, IsBdelta U <=>
    exists f g, IsScottContinuous f /\ IsScottContinuous g /\
      U = {x | f x = g x}
```

6.8 Theorems 7.1–7.4 — restricted equivalences

Ea a is the identity equivalence on the range of a ; it is isomorphic to the kernel equivalence `kerE a`. Theorem 7.2 packages the function-space, product, and sum identities, including the extra $\{\perp, \top\}$ summands on $\oplus$:

Lean 4 source

```
theorem theorem_7_2 {a b : Pomega} (ha : IsRetract a) (hb : IsRetract b) :
  (forall x y, funOf a x = funOf a y <=> (Ea a).rel (funOf a x) (funOf a y)) /\
  (forall f g, ... E_{a comp -> b} ...) /\
  (forall u v, (Ea (tensorR a b)).rel u v <=> (prodE (Ea a) (Ea b)).rel u v) /\
  (forall u v, (Ea (plusR a b)).rel u v <=>
    (sumE (Ea a) (Ea b)).rel u v /\
    (u = botElem /\ v = botElem) \/ (u = topElem /\ v = topElem))
```

Theorem 7.3 places I, K, S in the appropriate function spaces. Theorem 7.4 characterizes the iterators Z_n .

7 Source fidelity and remaining notes

Working source: [sources/Data_Types_as_Lattices_vision.md](#), a transcription of [sources/Data_Types_as_Lattices.pdf](#). See NOTICE and [sources/README.md](#) for copyright carve-outs.

Recorded divergences and limitations:

- Comparator selects the 63 theorem declarations and 70 locked definitions named in `comparator.json`. A match establishes those selected statements, not every informal clause of every numbered theorem.
- Compared substitution is `theorem_1_3` (unary composition), `theorem_1_3_tuple` (binary), and `theorem_1_3_nary` (ternary), not arbitrary finite-arity substitution.

- `theorem_1_5` proves continuity of `extend f` for every f , a strengthening of Scott’s hypothesis that f is continuous; `theorem_1_5_extends` restores that hypothesis for agreement on the subspace.
- Theorem 1.6 concerns countably based T_0 topological spaces, not countable partial orders with directed suprema.
- `theorem_2_2` is a conditional semantic congruence: it assumes the renamed bodies already have equal interpretations and lifts that equality through `lambda`. It is not source-level α -conversion from a freshness hypothesis.
- `theorem_4_6_limit` is an inclusion-preserving bijection with the inverse limit (an order isomorphism). Comparator does not check a homeomorphism.
- `theorem_6_5` is the specialization of Scott’s parameterized $\mathfrak{F} \cap \mathfrak{G}_\delta$ theorem to the representative target `succSet topElem`.
- `Ea a` is the identity equivalence on the range of a ; it is isomorphic to the kernel equivalence `kerE a`. The kernel relation is $\{(x, y) \mid a(x) = a(y)\}$.
- The locked coproduct `plusR` uses the doubly strict conditional `dcondSet` of (4.4), not the ordinary McCarthy conditional written in displayed (4.10). Mixed tags map to $\top$ rather than the union of the two injections.
- Theorem 4.5 packages the four numbered sum identities. Scott’s informal remark that the coproduct mediator is not unique is not a numbered theorem and is omitted.
- No external mathematical review has been performed. Every proof is checked by the Lean kernel, but the project remains self-assessed.

8 Palomar statement of record

`Challenge.lean` contains Mathlib-only declarations with deliberate theorem holes. `Solution.lean` imports the completed library. `comparator.json` compares the 63 selected theorem declarations and 70 locked definitions named in `comparator.json`.

File	Role
<code>Challenge.lean</code>	Mathlib-only statement of record with deliberate <code>sorry</code>
<code>Solution.lean</code>	Re-exports matching kernel-checked declarations
<code>comparator.json</code>	Compared theorem/definition names and permitted axioms
<code>formalization.yaml</code>	Scope, source alignment, fidelity, and review metadata
<code>PROVENANCE.md</code>	Standalone status and relation to sibling Scott projects

A Comparator match establishes only the selected `comparator.json` names. Further library companions are not all compared. Equation (4.45) is library-complete and is not a Palomar-compared name. `Solution.lean` and every file below `Scott1976/` must remain `sorry`-free.

9 Build and preflight

The repository pins Lean and mathlib **v4.33.0**.

```
lake exe cache get
lake build
bash scripts/palomar_preflight.sh --mechanical-only # CI / routine
bash scripts/palomar_preflight.sh # before Palomar submission
bash scripts/generate_arxiv_with_code.sh # -> arxiv_with_code.md
```

`lake build` checks the `Scott1976`, `Challenge`, and `Solution` targets. Mechanical preflight validates packaging, builds the project, compares Challenge/Solution declaration types and closure values, runs Palomar’s pinned Comparator, scans completed sources for proof holes, and checks permitted axioms.

The Appendix lists each library file with a short description and a link to the Palomar-preserved copy at <https://github.com/PalomarArchive/catskillsresearch-scott1976-4994ad81a666/tree/2081973> rather than reproducing the sources in full. `arxiv_with_code.md` is a generated review copy of this narrative (gitignored). Regenerate it with `scripts/generate_arxiv_with_code.sh` whenever `arxiv.md` changes.

10 License and source PDF

Original Lean code and author-written documentation are Apache-2.0. `sources/Data_Types_as_Lattices.pdf` and its transcription are not Apache-2.0; see NOTICE and `sources/README.md` for the copyright carve-out.

11 Conclusion

The library is a 1:1 onto translation of the numbered definitions, theorems, displayed equations used as definitions or examples, and Tables 1–3 of Scott’s 1976 monograph [Sco76]. Palomar compares 63 theorem declarations and 70 locked definitions; a Comparator match establishes those selected statements, not every informal clause of every numbered theorem. The one source-facing modification of a displayed equation is the locked coproduct `plusR`, which uses the doubly strict conditional of (4.4) rather than the ordinary conditional written in (4.10). Every completed proof is checked by the Lean kernel. The project remains self-assessed.

12 Acknowledgments

- **Dana Scott** — *Data Types as Lattices* [Sco76], the paper this development formalizes. Scott did not review or endorse the formalization.

12.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [COPE24]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`):

- **Cursor** [Cur26] — agent-assisted editing in the Cursor IDE: formalizing Scott’s 1976 *Data Types as Lattices* in Lean 4 / mathlib, `lake build` repair, vision-OCR transcription, drafting this narrative (`arxiv.md`), and tracking the formalized inventory. Generated Lean was provisional until it compiled under the pinned toolchain.

- **Cursor Composer 2.5 Fast [Cmp25]** — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott1976/DataTypesAsLattices/`, documentation, and medium proof obligations where the strategy was already fixed.
- **Anthropic Claude Sonnet 5 (medium reasoning) [Son26]** — day-to-day formalization and proof-engineering in Cursor at the medium reasoning tier: inventory and narrative maintenance, module wiring, and medium-complexity Lean obligations where the proof strategy was already fixed.
- **Anthropic Claude Opus 4.8 (high reasoning) [Ant26]** — selective use for the heaviest proof work. Every emitted proof term was checked by the Lean kernel.

All definitions and final prose were reviewed by the human author, who takes full responsibility for them.

12.2 Artifact availability

The development [ER76] is at github.com/catskillsresearch/scott1976. The Palomar-preserved copy linked in the Appendix is <https://github.com/PalomarArchive/catskillsresearch-scott1976-4994ad81a666/tree/2081973122a631d34dc74f83cfc3f7495991415b>.

13 References

- [Sco76] D. S. Scott. *Data Types as Lattices*. Technical Monograph PRG-5, Oxford University Computing Laboratory, Programming Research Group, September 1976; reprinted *SIAM Journal on Computing* 5 (1976), 522–587.
- [ER76] Lars Warren Ericson. *scott1976* (this work). <https://github.com/catskillsresearch/scott1976>.
- [COPE24] Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement*. 2024. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- [Cur26] Anywhere, Inc. *Cursor: AI-native code editor and agent environment*. <https://cursor.com> (accessed 2026).
- [Cmp25] Anywhere, Inc. *Composer 2.5*. Model announcement and documentation, <https://cursor.com/blog/composer-2-5>; model card as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Son26] Anthropic. *Claude Sonnet 5* (medium reasoning variant). System card, <https://www.anthropic.com/claude-sonnet-5-system-card>; model documentation as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Ant26] Anthropic. *Claude Opus 4.8* (high thinking/reasoning variant). System card and announcement, <https://www.anthropic.com/news/claude-opus-4-8>; model documentation as integrated in Cursor, <https://cursor.com/docs/models/claude-opus-4-8> (accessed 2026).

A Lean sources in the Palomar archive

The complete Lean sources are not reproduced in this note. Each library file below is the copy preserved by Palomar at commit `2081973122a631d34dc74f83cfc3f7495991415b`. The archive tree is <https://github.com/PalomarArchive/catskillsresearch-scott1976-4994ad81a666/tree/2081973122a631d34dc74f83cfc3f7495991415b>. The working repository is <https://github.com/catskillsresearch/scott1976>.

A.1 `Scott1976.lean`

Root import module. It loads the fourteen `DataTypesAsLattices` modules in dependency order so that `import Scott1976` typechecks the complete sorry-free development.

A.2 `Scott1976/DataTypesAsLattices/Pomega.lean`

§1 carrier: $\mathbf{P}\omega$ as `Set ℕ`, bottom and top, the finite basis e_n from binary expansions, pairing, and the elementary lattice operations.

A.3 `Scott1976/DataTypesAsLattices/Continuous.lean`

§1 Scott topology: basic neighbourhoods $\{x \mid e_n \subseteq x\}$, Scott-open sets, continuity $f(x) = \bigcup f(e_n)$ on $e_n \subseteq x$, and Theorem 1.1.

A.4 `Scott1976/DataTypesAsLattices/Graph.lean`

§1 graphs of continuous maps: `graph` / `funOf`, application in the graph model, and Theorem 1.2 (u is a graph iff $u = \text{graph}(\text{fun } u)$).

A.5 `Scott1976/DataTypesAsLattices/FixedPoint.lean`

§1 substitution and least fixed points: Theorems 1.3–1.4, the iterates $f^n(\emptyset)$, and the least-fixed-point combinator Y .

A.6 `Scott1976/DataTypesAsLattices/Embedding.lean`

§1 extension and embedding: the maximal continuous extension `extend f` (Theorem 1.5) and the embedding of countably based T_0 spaces into $\mathbf{P}\omega$ (Theorem 1.6).

A.7 `Scott1976/DataTypesAsLattices/Lambda.lean`

§2 LAMBDA: Table 1–2, combinators, the inductive syntax and interpretation `interp`, displayed equations (2.1)–(2.28), and Theorems 2.1–2.5.

A.8 `Scott1976/DataTypesAsLattices/Computability.lean`

§2 computability: recursively enumerable graphs, combinatory and closed LAMBDA-definable elements, and Theorem 2.6 in unary and k -ary form.

A.9 `Scott1976/DataTypesAsLattices/Enumeration.lean`

§3 enumeration: the combinator G , Scott’s `val` / `apply` / `op` / `arg`, Theorems 3.1–3.7, and the tree evaluator `vaal` of (4.46).

A.10 `Scott1976/DataTypesAsLattices/FixedLattices.lean`

Order-theoretic support used later by retracts and closures: directed sets, compactness, and the countable algebraic and continuous lattice interfaces.

A.11 `Scott1976/DataTypesAsLattices/Retracts.lean`

§4 retracts: idempotent graphs, typedness, the constructors $\circ \rightarrow / \otimes / \oplus$, Theorems 4.1–4.6, inverse limits, and the recursive domains `int`, `tree`, `lamb`, and `exp`.

A.12 `Scott1976/DataTypesAsLattices/Closures.lean`

§5 closures: $I \subseteq a = a \circ a$, pairing and the boxed product and sum, the universe V , and Theorems 5.1–5.6.

A.13 `Scott1976/DataTypesAsLattices/Classification.lean`

§6 classification of subsets of $\mathbf{P}\omega$: the classes $\mathfrak{G}$, $\mathfrak{F}$, $\mathfrak{B}$ and their δ -refinements, Theorems 6.1–6.7, and the typical sets of Table 3.

A.14 `Scott1976/DataTypesAsLattices/Functionality.lean`

Remaining §4 identities (typed $\otimes/\oplus$, CCC `eval/curry`, injections), the continuous decoder $\mathcal{H}$ of (4.45), and §7 restricted equivalences (Theorems 7.1–7.4).

A.15 `Scott1976/DataTypesAsLattices/Basic.lean`

Thin library facade. It re-exports `Enumeration`, `Classification`, and `Closures` so that `Solution.lean` can import the completed development in one step.